Renta Form

Documentación funcional con capturas de pantalla
De principio a fin

Generado el 26 de abril de 2026

Índice

1. Introducción
2. Página de aterrizaje del formulario público
3. Datos de identificación rellenos
4. Asistente de preguntas Sí / No (paso 1 de las preguntas)
5. Avance entre preguntas
6. Cambio de idioma en caliente
7. Confirmación de envío
8. Documentación interactiva de la API (Swagger UI)
9. Acceso al panel de administración
10. Login con credenciales
11. Pestaña "Declaraciones"
12. Pestaña "Preguntas"
13. Pestaña "Usuarios"
14. Pestaña "Idiomas"
15. Pestaña "Traducciones"
16. Modelo de datos
17. Puesta en marcha (resumen)
18. Estado verificado

1. Introducción

Renta Form es una aplicación web para la gestión de declaraciones de la Renta (IRPF). Está dividida en dos áreas principales:

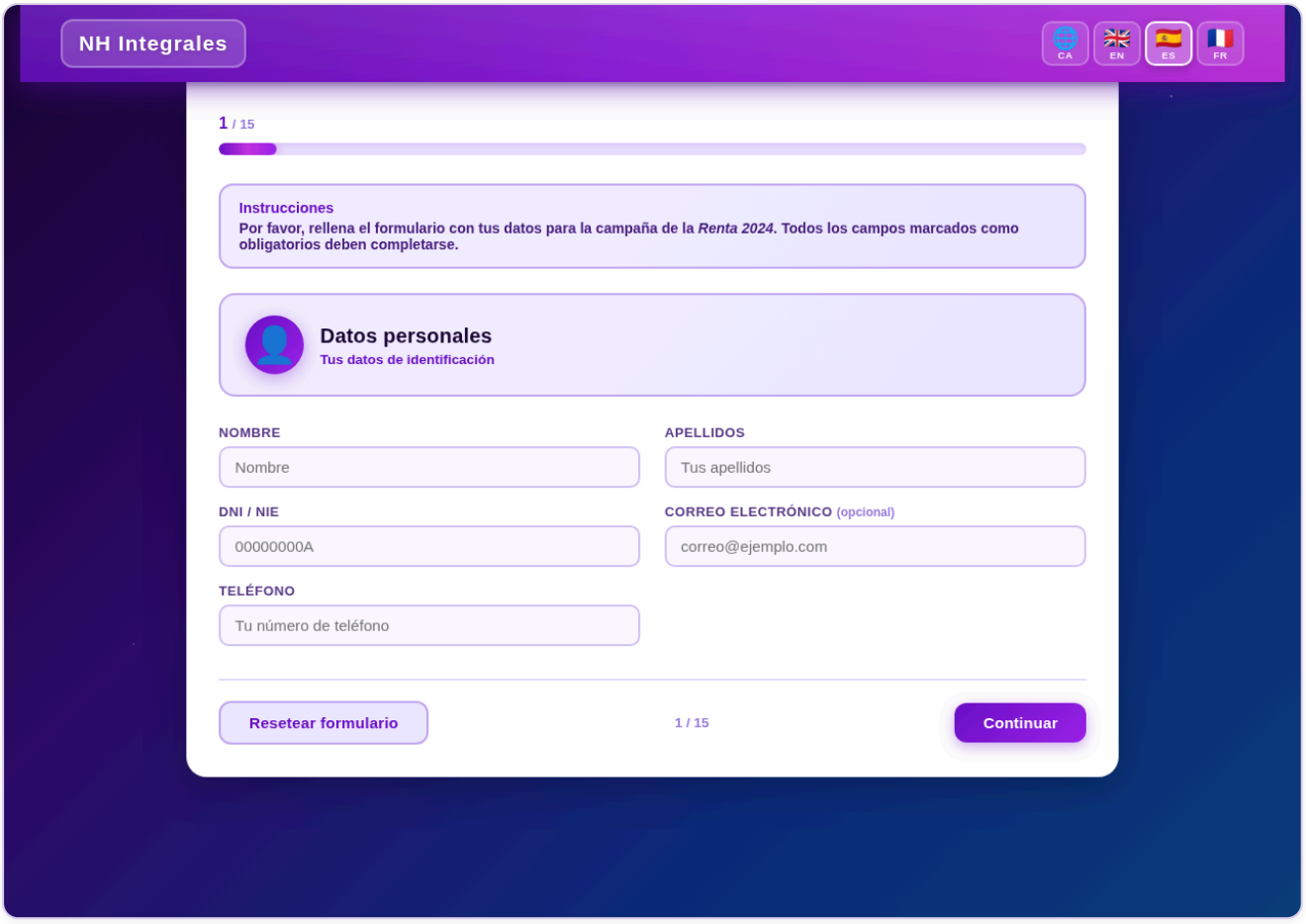
- **Formulario público** – un asistente paso a paso (tipo "quiz") en el que el ciudadano introduce sus datos identificativos y responde a un conjunto de preguntas Sí / No sobre su situación fiscal.
- **Panel de administración** – un back-office protegido por usuario y contraseña para gestionar las declaraciones recibidas, los usuarios, el catálogo de preguntas y la internacionalización (idiomas y traducciones).

Tecnológicamente la aplicación se compone de tres capas:

Capa	Tecnología	Puerto local
Frontend	React 19 + Vite	5173 (dev) / 3001 (build servida por backend)
Backend	Node.js + Express	3001
Base de datos	PostgreSQL 16	5432

La especificación de la API se publica en `/#/api-docs` mediante Swagger UI y todas las rutas REST cuelgan del prefijo `/v1`.

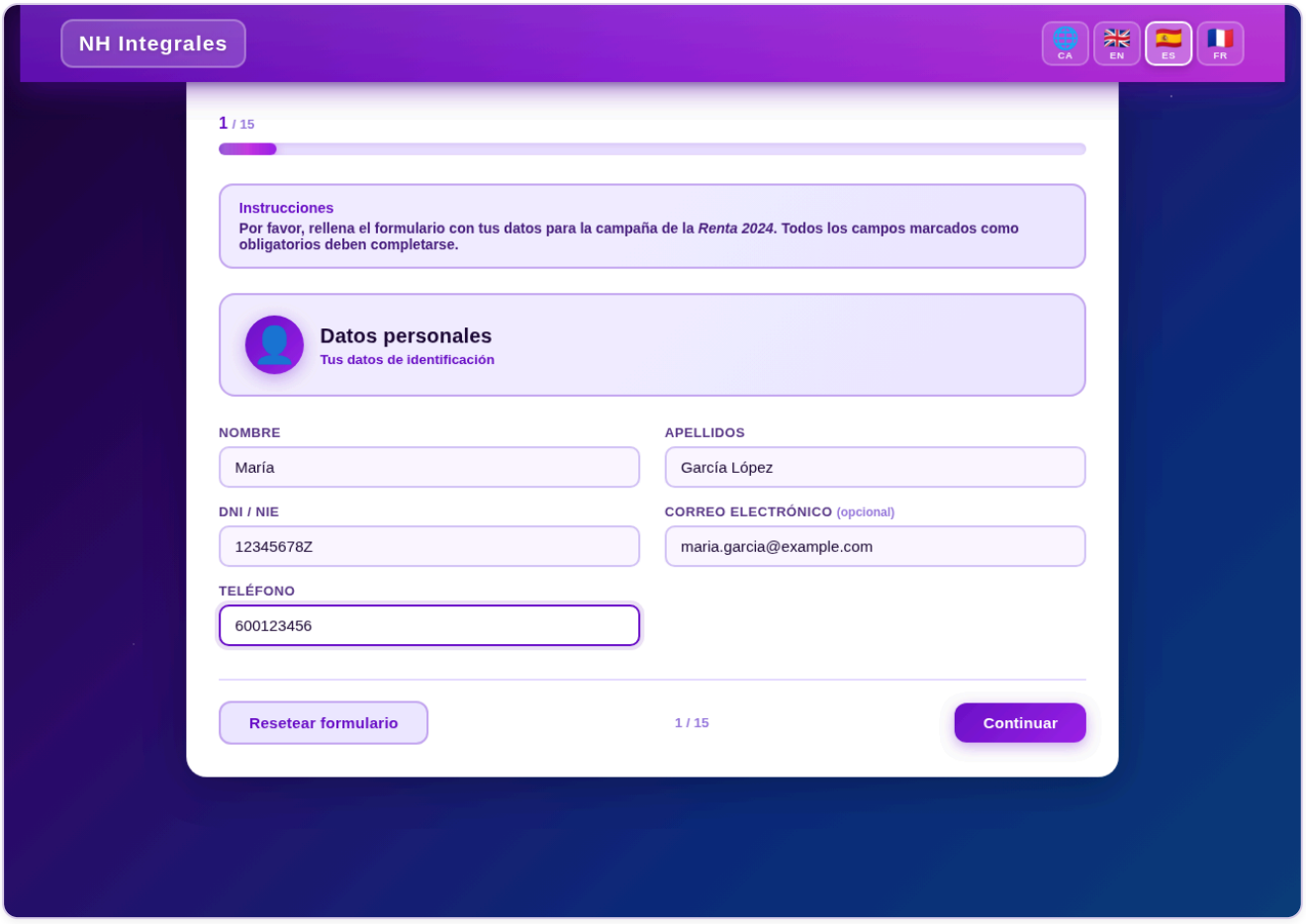
2. Página de aterrizaje del formulario público



Al acceder a la raíz (`http://localhost:3001/`) el usuario aterriza en el primer paso del asistente. La cabecera contiene el logotipo de la aplicación y un selector de idioma con bandera (ES, CA, EN, FR). La barra superior muestra el progreso (paso 1 de 15) y una caja de instrucciones describe el propósito de la campaña.

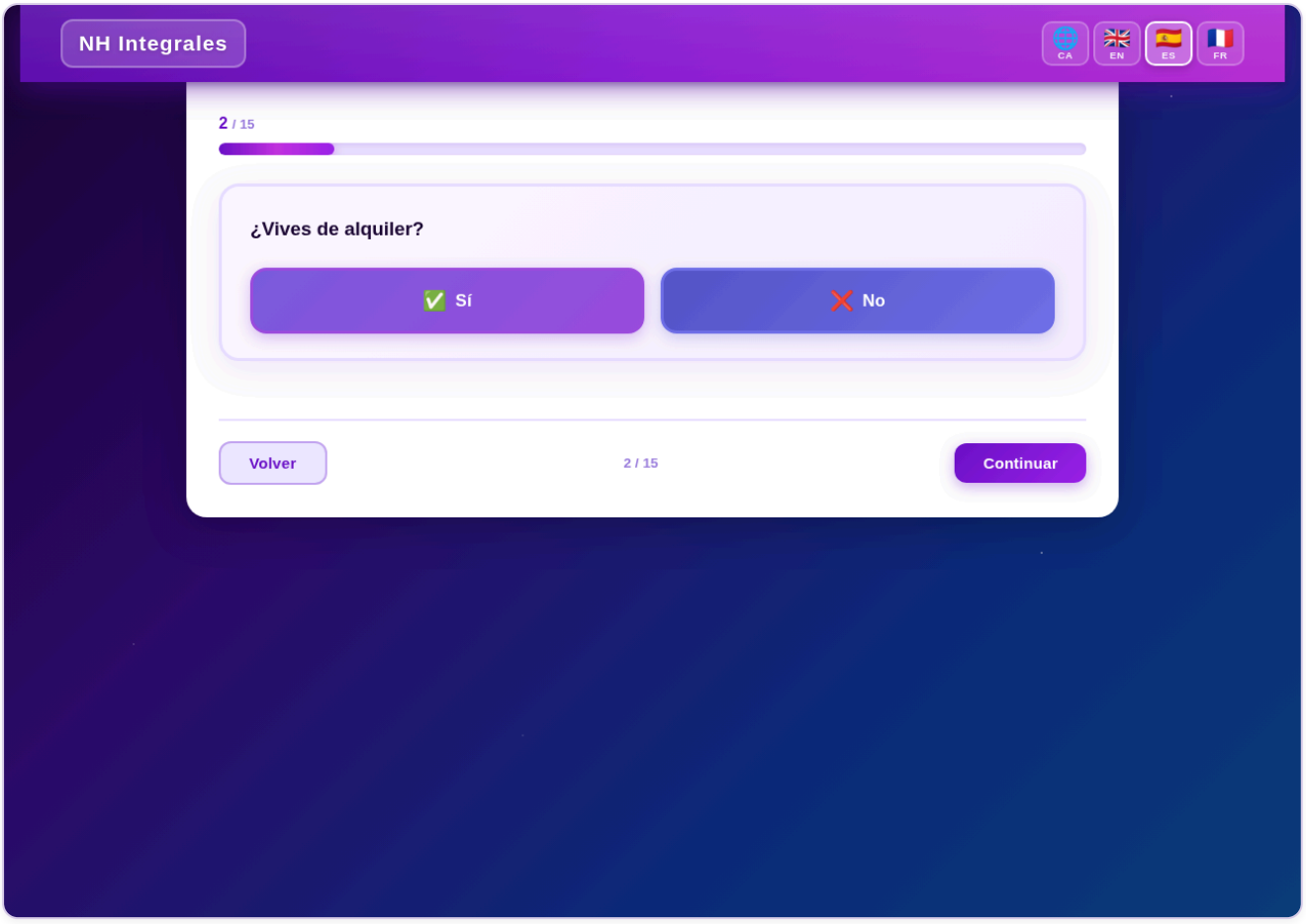
El primer paso pide los datos identificativos: *Nombre, Apellidos, DNI / NIE, Email (opcional) y Teléfono*. Hasta que se rellenen los campos obligatorios, el botón **Continuar** bloquea el avance.

3. Datos de identificación rellenos



Los campos validan formato (DNI/NIE de 9 caracteres, email opcional con formato válido). El campo **DNI/NIE** es la clave única de la declaración: si ya existe una declaración con ese DNI, el backend devolverá un error de duplicado.

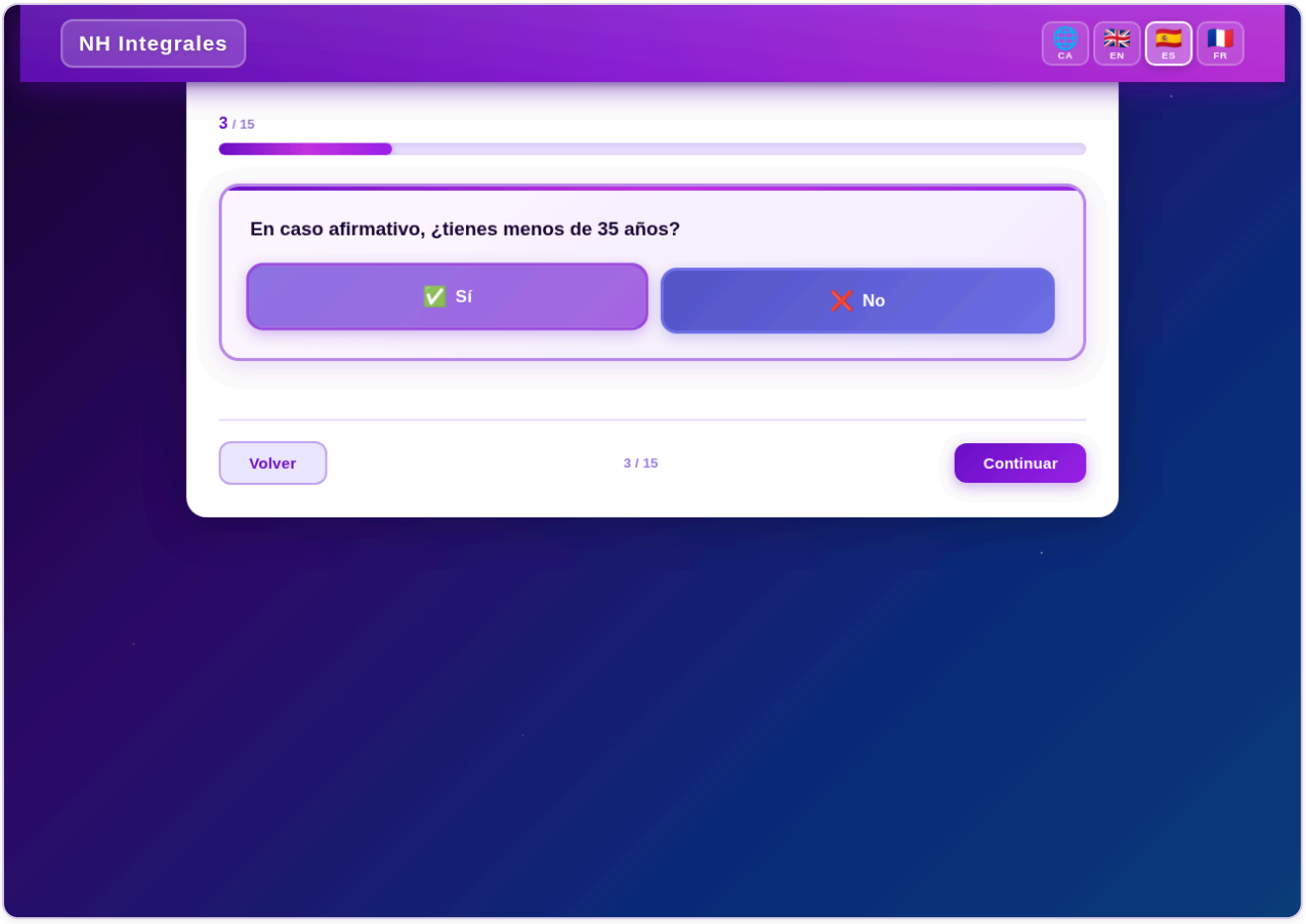
4. Asistente de preguntas Sí / No (paso 1 de las preguntas)



A partir del paso 2 cada pantalla muestra **una única pregunta** en formato tipo quiz. Las preguntas se cargan desde la base de datos (tabla `preguntas`) y cubren cuatro bloques: situación de la vivienda, cargas familiares y ayudas, hijos a cargo e ingresos extraordinarios e inversiones.

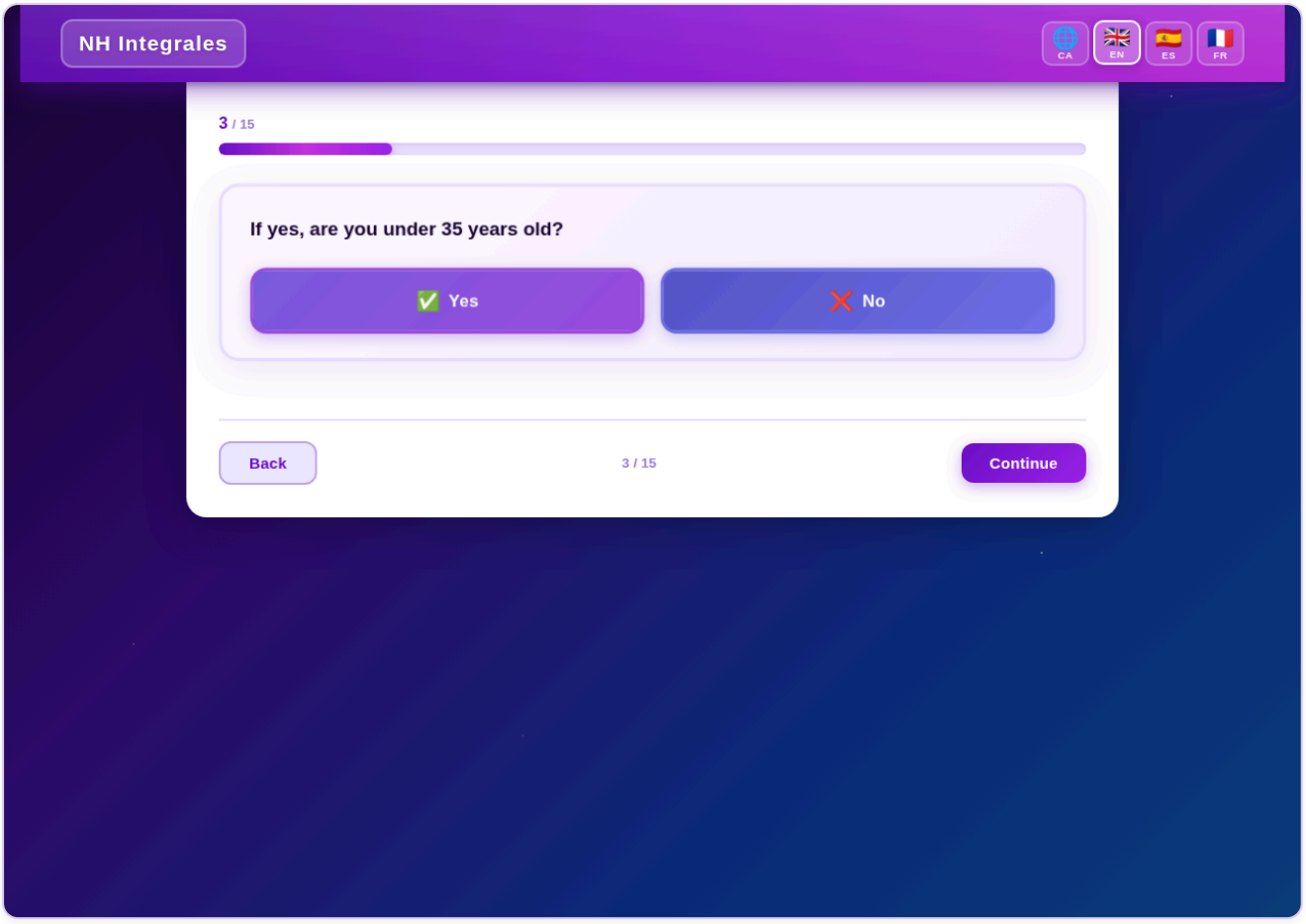
El catálogo actual contiene **14 preguntas**, todas con dos respuestas posibles: **Sí** o **No**. Al elegir una respuesta se muestra un anillo de feedback animado y el asistente avanza automáticamente al siguiente paso.

5. Avance entre preguntas



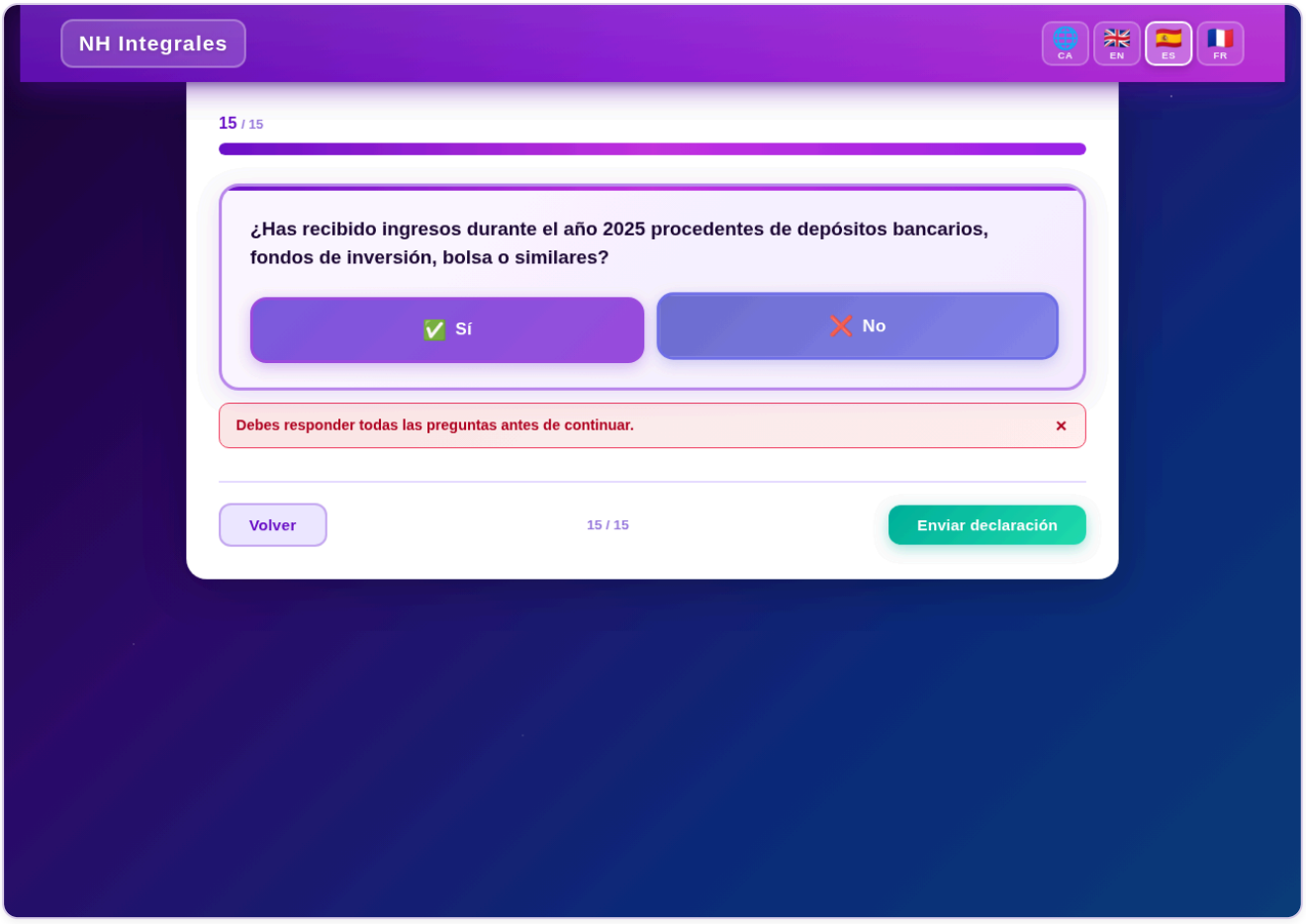
La barra superior y el contador *"X / 15"* reflejan en tiempo real el progreso. El botón **Volver** permite revisar respuestas anteriores y los datos se conservan en memoria mientras el usuario navega.

6. Cambio de idioma en caliente



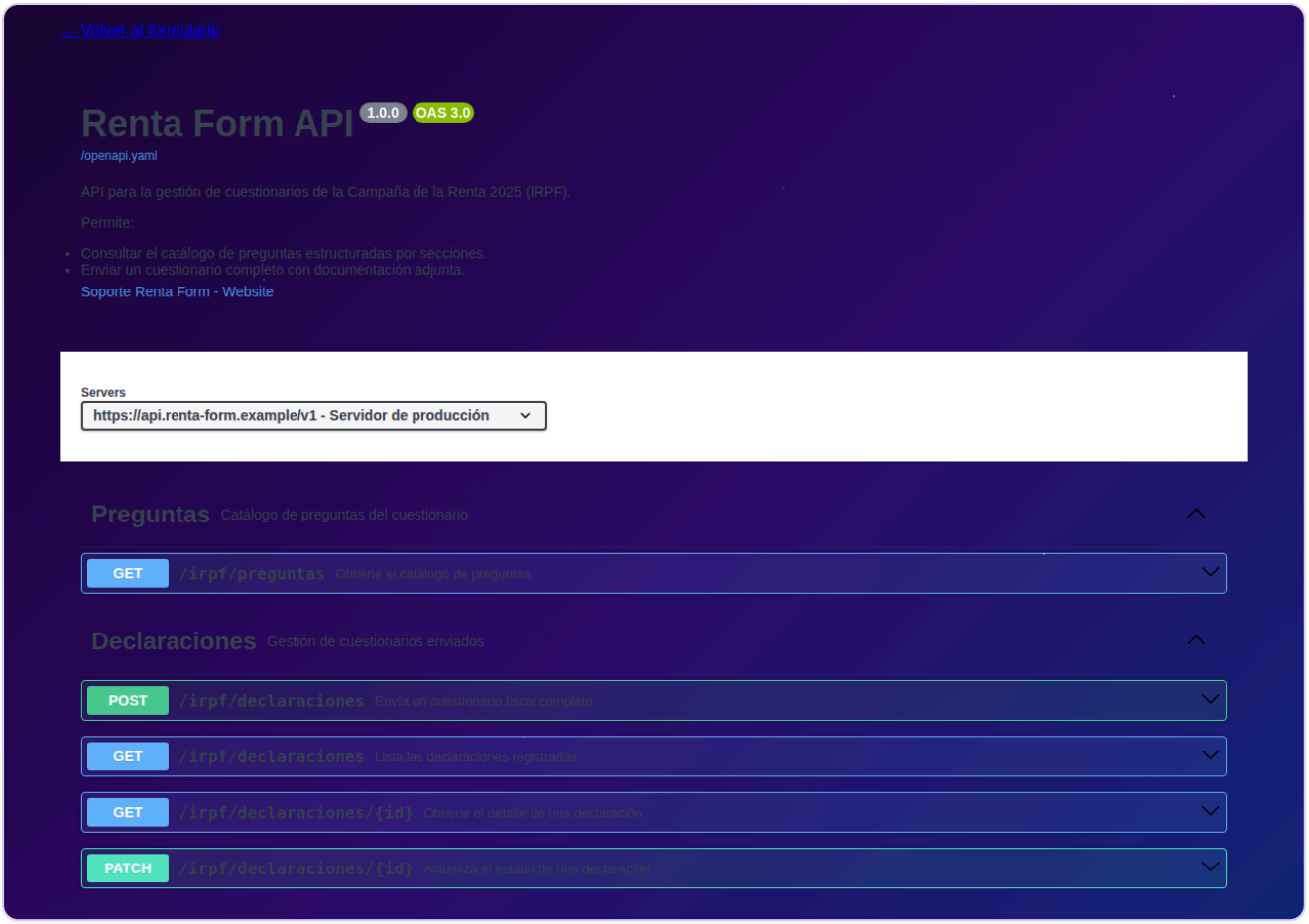
Pulsando una de las banderas de la cabecera, todos los textos del asistente se traducen al instante (ES, CA, EN, FR). Los idiomas disponibles y sus traducciones se obtienen de la base de datos a través de los endpoints públicos `GET /v1/irpf/idiomas` y `GET /v1/irpf/traducciones`, por lo que el administrador puede dar de alta nuevos idiomas y editar textos sin necesidad de volver a desplegar la aplicación.

7. Confirmación de envío



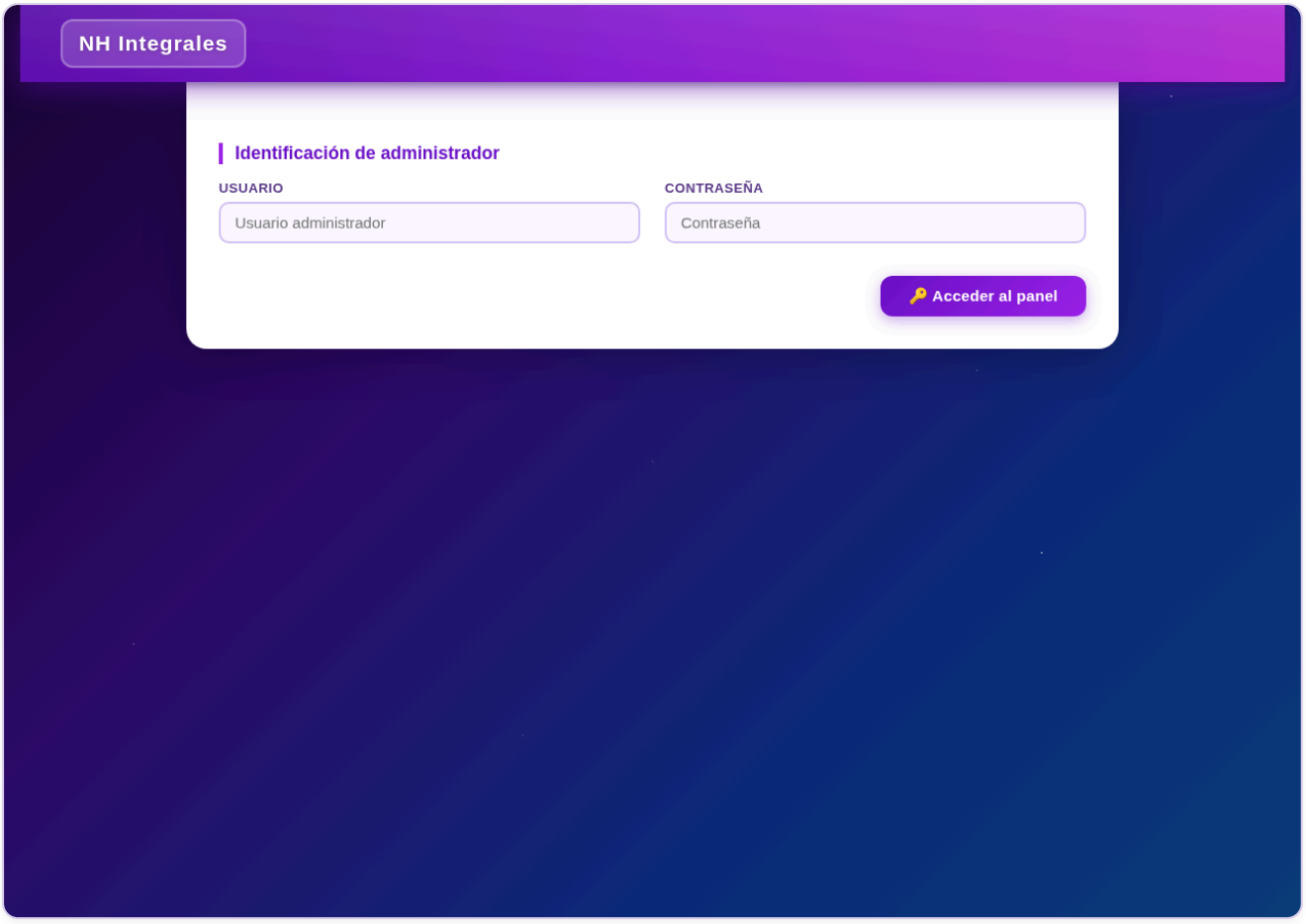
Al responder a la última pregunta y pulsar **Enviar declaración**, el backend almacena un nuevo registro en la tabla `declaraciones` con estado inicial *recibido*. El usuario ve una pantalla de éxito con animación de confeti y la opción **Nueva declaración** para empezar otra.

8. Documentación interactiva de la API (Swagger UI)



En `/#/api-docs` se sirve Swagger UI con la especificación OpenAPI completa. Desde aquí se pueden inspeccionar y probar todos los endpoints: autenticación (`/auth/login`, `/auth/admin-login`), preguntas (`/irpf/preguntas`), declaraciones (`/irpf/declaraciones`), idiomas y traducciones (`/irpf/idiomas`, `/irpf/traducciones`) y la zona de administración (`/admin/...`).

9. Acceso al panel de administración

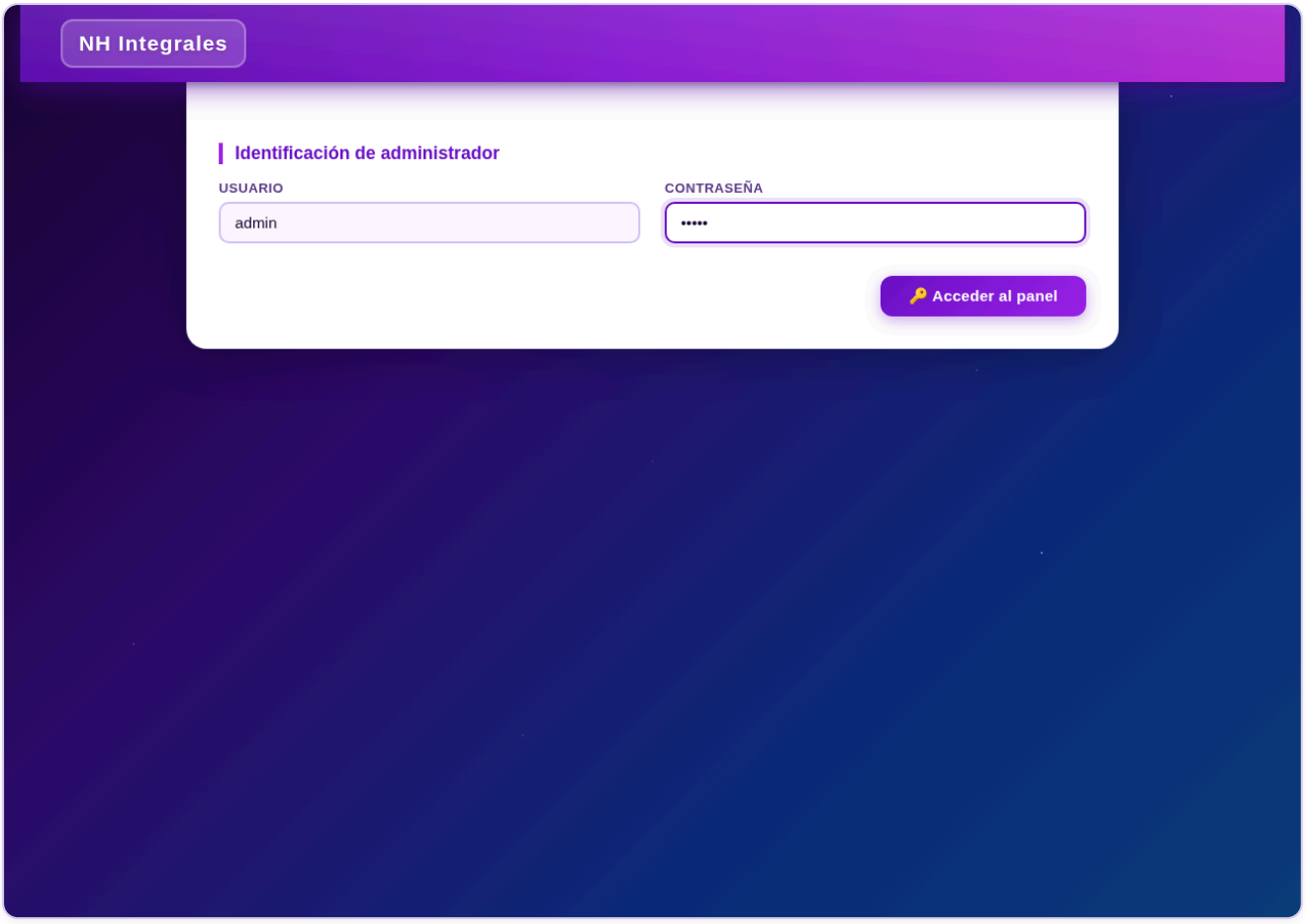


El back-office se encuentra en `http://localhost:3001/#/backend_admin`. Si el navegador no tiene una sesión activa de administrador, se muestra una pantalla de login. Las credenciales por defecto del seed son:

Campo	Valor
Usuario (DNI)	`admin`
Contraseña	`admin`

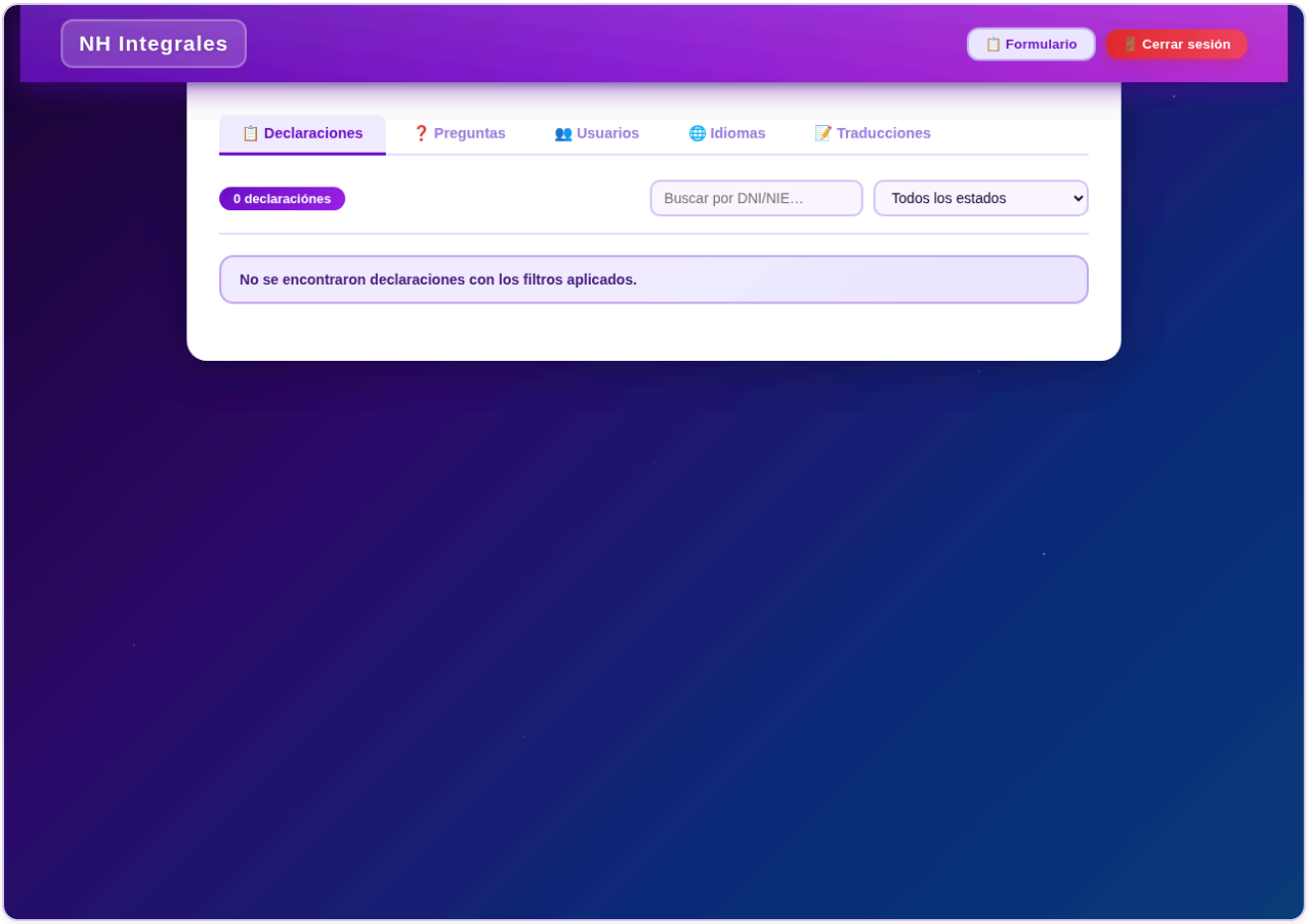
Estas credenciales se siembran con el script `npm run db:seed-admin` y pueden personalizarse con las variables `ADMIN_DNI`, `ADMIN_EMAIL` y `ADMIN_PASSWORD`.

10. Login con credenciales



El formulario envía las credenciales a `POST /v1/auth/admin-login`. El backend valida la contraseña con **bcrypt** contra la tabla `usuarios`, comprueba que el usuario tenga rol *admin* y devuelve los datos de sesión que el frontend guarda en su `AuthContext`.

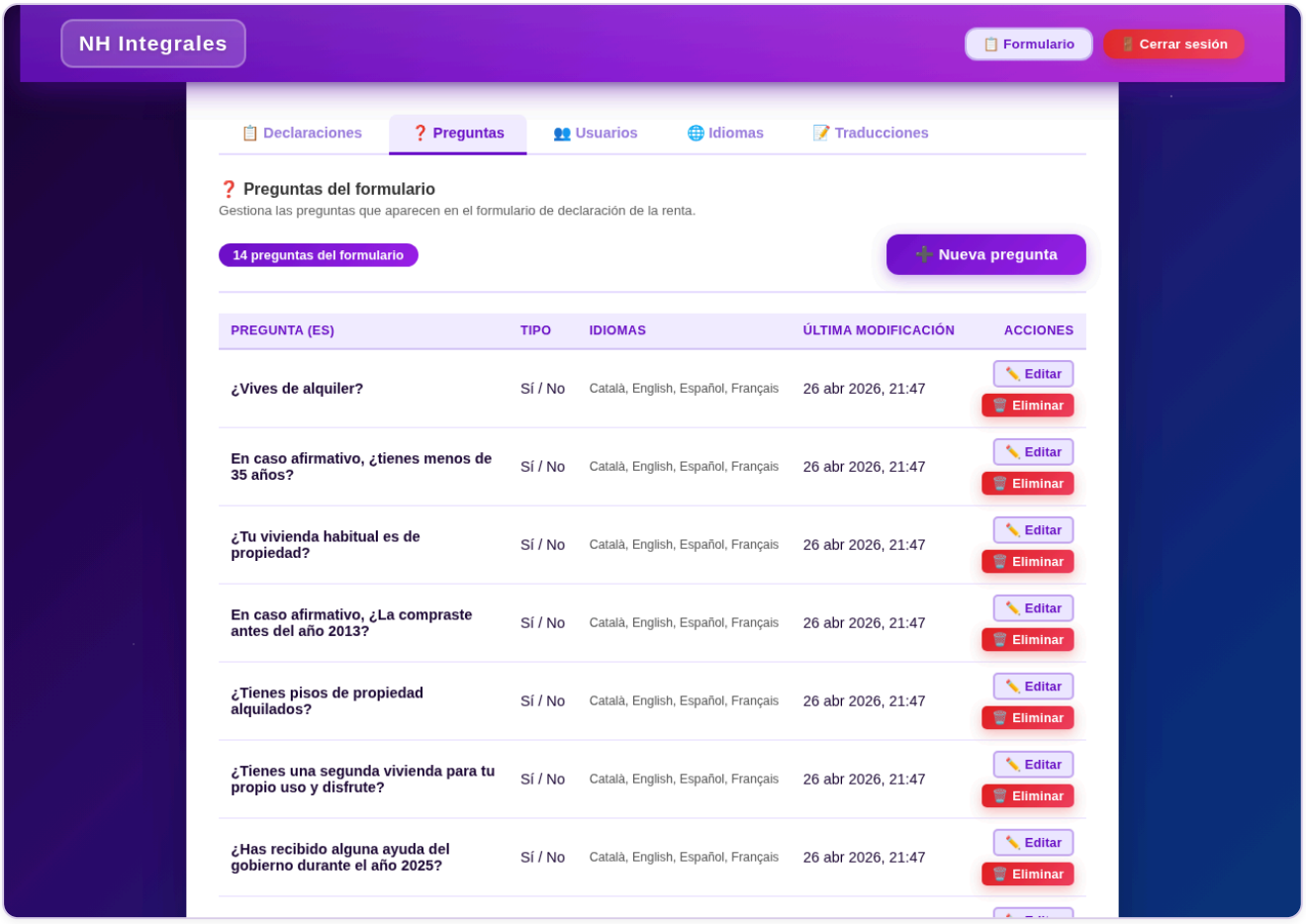
11. Pestaña "Declaraciones"



Es la vista por defecto al entrar al panel. Lista todas las declaraciones IRPF almacenadas en la base de datos con paginación, filtro por estado y búsqueda por DNI/NIE. Sobre cada declaración el administrador puede:

- Cambiar el **estado**: *recibido* → *en revisión* → *documentación pendiente* → *completado* → *archivado*.
- **Editar** los datos identificativos y las respuestas a las preguntas.
- **Eliminar** la declaración.
- **Asignar** una cuenta de usuario al ciudadano (búsqueda por DNI).
- **Subir un PDF** de la renta firmada y **descargar** un PDF generado dinámicamente.

12. Pestaña "Preguntas"

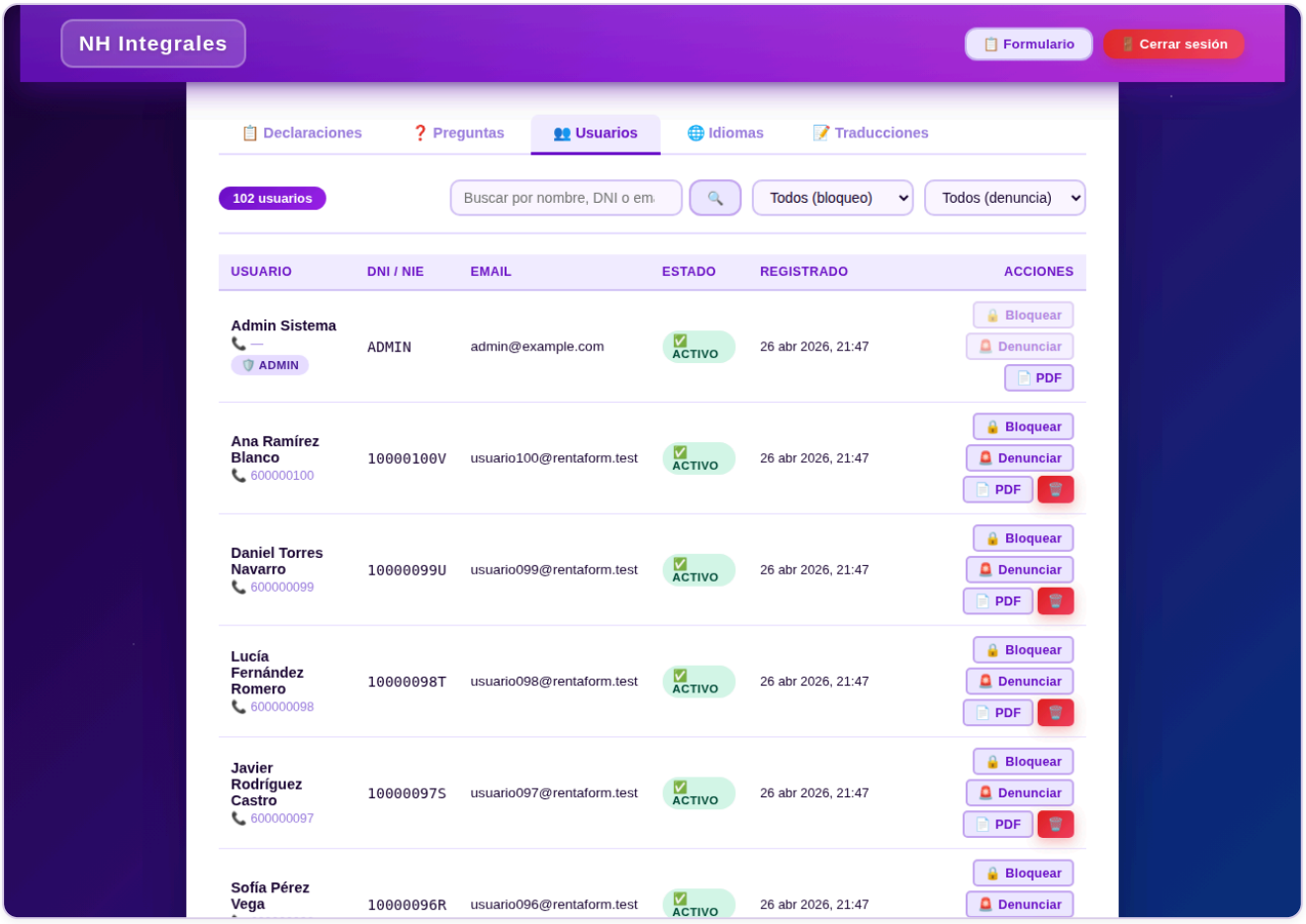


CRUD del catálogo de preguntas mostradas en el formulario público. Cada pregunta tiene:

- Un **campo** técnico (clave estable, p. ej. `viviendaAlquiler`) que se corresponde con una columna de la tabla `declaraciones`.
- Un **texto multiidioma** almacenado como JSONB (`{ es: "...", en: "...", ca: "...", fr: "..." }`) con fallback al castellano.
- Una **sección** y un **orden** que controlan cómo se agrupan en el asistente.

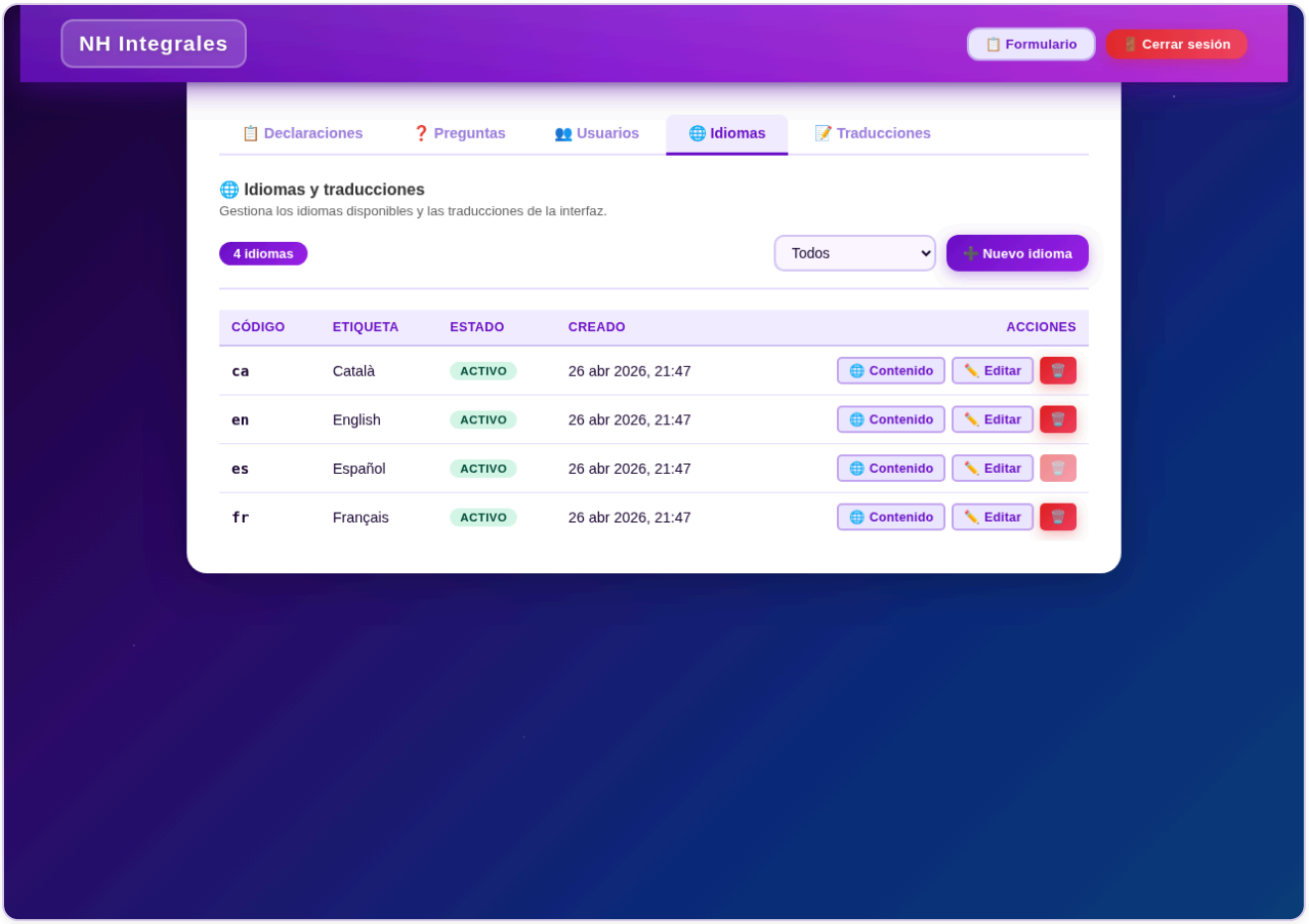
Solo las preguntas cuyo `campo` coincide con una columna real de la tabla `declaraciones` se muestran al ciudadano (filtro `QUESTION_CAMPOS` en el frontend).

13. Pestaña "Usuarios"



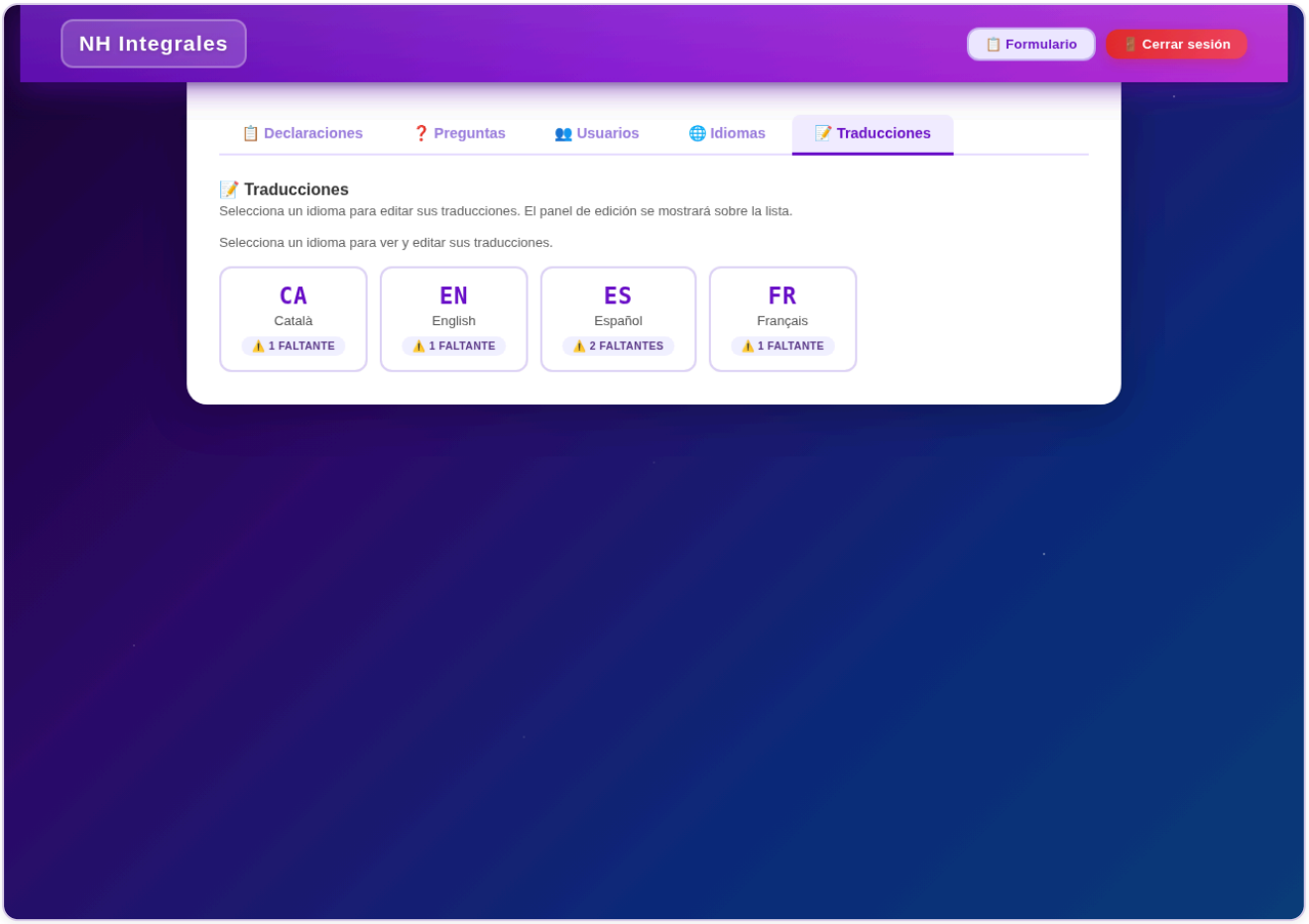
Gestión del padrón de cuentas. Permite ver los 100 usuarios sembrados por defecto, bloquear / desbloquear cuentas, marcar denuncias y asignar las preguntas que cada usuario debe responder. La contraseña se almacena *hashed* con bcrypt y nunca se muestra en claro.

14. Pestaña "Idiomas"



Permite dar de alta y dar de baja idiomas (tabla `idiomas`: *code, label, activo*). Solo los marcados como activos aparecen en el selector de bandera del frontend público. Desde esta pestaña el administrador también puede subir o descargar el contenido completo de un idioma vía los endpoints `/v1/admin/idiomas/{id}/content`.

15. Pestaña "Traducciones"



Editor en tabla de la lista completa de claves de traducción (alrededor de 100 claves) y sus valores en cada idioma (tabla `traducciones`: *idioma_id, clave, valor*). Cualquier cambio se refleja inmediatamente en el formulario público al recargar las traducciones.

16. Modelo de datos

El esquema (definido en `database/init.sql`) consta de cinco tablas:

1. **usuarios** – cuentas de ciudadanos y administradores; almacena DNI/NIE único, email, contraseña hasheada, rol y banderas de bloqueo / denuncia.
2. **preguntas** – catálogo de preguntas con campo único, texto JSONB multiidioma, sección y orden.
3. **declaraciones** – una fila por declaración; columnas con datos identificativos y una columna por respuesta. `dni_nie` es *UNIQUE*.
4. **idiomas** – idiomas activos del sistema.
5. **traducciones** – pares clave/valor por idioma.

17. Puesta en marcha (resumen)

Pasos para arrancar la aplicación en local:

```
docker-compose up -d                # PostgreSQL en :5432
npm install
cd backend && npm install && cd ..
cp backend/.env.example backend/.env
npm run db:setup-all                # migra esquema + siembra datos
npm run start:backend               # backend en :3001
npm run dev                         # frontend en :5173
```

Tests E2E con Cucumber + Playwright: `npm run test`. Linter: `npm run lint`.

18. Estado verificado

Durante la captura de este documento se ha comprobado que **todos los flujos descritos están operativos** sobre el build de producción servido por el backend en el puerto 3001:

- ✅ Carga de idiomas y traducciones desde la base de datos.
- ✅ Asistente público completo: validación de identidad, navegación entre preguntas Sí/No, cambio de idioma en caliente y envío final con persistencia en `declaraciones`.
- ✅ Pantalla de éxito con confeti tras enviar la declaración.
- ✅ Documentación de la API (Swagger UI) accesible en `/#/api-docs`.
- ✅ Login de administrador con `admin / admin` (hash bcrypt en BD).
- ✅ Pestañas del panel admin: Declaraciones, Preguntas, Usuarios, Idiomas y Traducciones — todas se cargan y muestran datos sembrados (100 usuarios, 14 preguntas, 4 idiomas, ~100 claves de traducción × 4 idiomas).